

BÀI 3. RAZOR VIEWS, LAYOUT & ROUTING

Đây là tuần học mà ứng dụng của chúng ta sẽ thực sự "hiện hình". Chúng ta sẽ học cách biến những dữ liệu trừu tượng thành một giao diện web đẹp mắt, nhất quán và dễ sử dụng.

MỤC TIÊU BÀI HỌC

Sau khi hoàn thành tuần học này, bạn sẽ có khả năng:

1. **Viết code C# bên trong HTML** một cách thành thạo bằng cú pháp Razor.
2. **Lựa chọn và sử dụng** phương pháp truyền dữ liệu từ Controller sang View một cách chuyên nghiệp.
3. **Thiết kế một layout chung** để tái sử dụng trên toàn bộ trang web, giúp tiết kiệm công sức và đảm bảo tính nhất quán.
4. **Giải thích và cấu hình** được hệ thống Routing của ASP.NET Core để tạo ra các URL thân thiện.

NỘI DUNG LÝ THUYẾT

3.1. Giới thiệu Razor Engine - Ngôn ngữ lai C# và HTML

3.1.1. Razor là gì?

Razor không phải là một ngôn ngữ lập trình mới. Nó là một **template engine** (công cụ tạo mẫu) với cú pháp cực kỳ tinh gọn, cho phép bạn **nhúng code C# trực tiếp vào trong các file HTML**. Nhiệm vụ của nó là kết hợp dữ liệu động (từ C#) với cấu trúc tĩnh (của HTML) để tạo ra một trang web hoàn chỉnh trước khi gửi về cho trình duyệt.

3.1.2. Cú pháp cơ bản - Sức mạnh của ký tự @

Ký tự @ là "cánh cổng ma thuật" để chuyển từ chế độ HTML sang chế độ C#.

- **Biểu thức đơn lẻ:** Dùng @ ngay trước một biến hoặc một lời gọi phương thức. Razor đủ thông minh để biết biểu thức kết thúc ở đâu.

```
<p>Chào bạn, hôm nay là: @DateTime.Now.ToShortDateString()</p>  
<p>Tên sản phẩm: @Model.Name</p>
```

- **Khối lệnh:** Dùng @ kết hợp với cặp ngoặc nhọn {...} để viết nhiều dòng code C#.

```
@{
    // Đây là một khối code C# bên trong file HTML
    var fullName = "Nguyễn Văn A";
    var yearOfBirth = 1995;
    var age = DateTime.Now.Year - yearOfBirth;
}
<p>Tên của bạn là: @fullName</p>
<p>Tuổi của bạn là: @age</p>
```

- **Vòng lặp và điều kiện:** Kết hợp @ với các cấu trúc điều khiển của C#.

```
@if (Model.Products.Count() > 0) {
    <ul>
        @foreach (var p in Model.Products) {
            <li>@p.Name - Giá: @p.Price.ToString("c")</li>
        }
    </ul>
} else {
    <p>Không có sản phẩm nào để hiển thị.</p>
}
```

3.1.3. Truyền dữ liệu: Controller nói chuyện với View như thế nào?

Khi người dùng yêu cầu một trang web, Controller là nơi xử lý logic (ví dụ: vào Database lấy danh sách sản phẩm). Nhưng Controller không có khả năng hiển thị HTML. Nó phải tìm cách "**gửi bưu kiện**" chứa dữ liệu đó sang cho View. Chúng ta có 3 cách chính:

1. ViewData (Gửi bằng Từ điển)

ViewData là một dạng Dictionary (Từ điển). Bạn nhét dữ liệu vào theo một "Chìa khóa" (Key) dạng chuỗi, và lấy ra ở View bằng đúng Key đó.

- Trong Controller:

```
public IActionResult Index()
{
    ViewData["ThongBao"] = "Chào mừng bạn đến với cửa hàng!";
    ViewData["SoLuong"] = 100;
    return View();
}
```

- Trong View (Index.cshtml):

```
<h2>@ViewData["ThôngBao"]</h2>
<p>Chúng tôi hiện có @ViewData["SoLuong"] sản phẩm.</p>
```

- *Nhược điểm:* Dễ gõ sai chính tả tên Key (ví dụ gõ ViewData["ThôngBao"] thì nó sẽ không hiện gì mà cũng không báo lỗi lúc code).

2. ViewBag (Anh em sinh đôi của ViewData)

ViewBag thực chất là một "lớp vỏ bọc" của ViewData sử dụng tính năng dynamic của C#. Thay vì dùng ngoặc vuông và chuỗi ["..."], bạn gọi nó như một thuộc tính.

- Trong Controller:

```
ViewBag.ThongBao = "Chào mừng bạn!";
ViewBag.SoLuong = 100;
```

- Trong View:

```
<h2>@ViewBag.ThongBao</h2>
<p>Số lượng: @ViewBag.SoLuong</p>
```

- *Nhược điểm:* Giống ViewData, không được Visual Studio hỗ trợ gợi ý code (IntelliSense) và chỉ báo lỗi khi web đang chạy (Runtime).

3. Strongly-Typed Model (Cách chuyên nghiệp - Khuyến dùng)

Đây là cách an toàn và mạnh mẽ nhất. Bạn đóng gói toàn bộ dữ liệu vào một Class (gọi là Model hoặc ViewModel), sau đó truyền thẳng Object đó cho View.

- Trong Controller:

```
var product = new Product { Name = "Laptop", Price = 1500 };
return View(product); // Truyền trực tiếp object vào phương thức View()
```

- Trong View:

```
@* Khai báo kiểu dữ liệu ở đầu file *@
@model YourProject.Models.Product

@* Lúc này, khi gõ @Model. , Visual Studio sẽ tự động xổ ra Name và Price *@
<h2>Tên: @Model.Name</h2>
<p>Giá: @Model.Price</p>
```

3.2. Layout, Partial Views và Tag Helpers - Xây dựng từ các mảnh ghép

3.2.1. Layout (_Layout.cshtml) - Khung nhà chung

Hãy tưởng tượng bạn xây một ngôi nhà. Bạn sẽ không xây móng, tường, mái riêng cho từng phòng. Bạn xây một bộ khung chung trước. _Layout.cshtml chính là **bộ khung chung** đó.

- **Vị trí:** Thường được đặt trong thư mục Views/Shared.
- **Chức năng:** Chứa tất cả các thành phần HTML lặp lại trên mọi trang (hoặc hầu hết các trang), ví dụ: <html>, <head>, <header>, <footer>, các link CSS, JS.
- Thành phần ma thuật - @RenderBody():
 - Đây là "**không gian trống**" mà Layout dành ra.
 - Nội dung của các View cụ thể (như Index.cshtml, About.cshtml) sẽ được "nhúng" vào đúng vị trí này.
- Nâng cao - @RenderSection(name, required):
 - Cho phép bạn định nghĩa các "ổ cắm" tùy chọn.
 - Ví dụ, trong Layout bạn có thể đặt @RenderSection("Scripts", required: false) ở cuối thẻ <body>.
 - Trong một View cụ thể cần script riêng, bạn có thể định nghĩa:

```
@section Scripts {  
    <script src="my-custom-script.js"></script>  
}
```

3.2.2. Chia để trị với Partial View

Nếu bạn nhét tất cả mã HTML của một trang (ví dụ: Trang chủ gồm Slider, Danh sách sản phẩm nổi bật, Danh sách bài viết mới, Form đăng ký email) vào duy nhất file Index.cshtml, file đó sẽ dài hàng ngàn dòng và cực kỳ khó bảo trì. Giải pháp là **Partial View** (Giao diện từng phần).

1. Partial View là gì?

Nó đơn giản là một file .cshtml nhỏ, chứa một đoạn mã HTML/Razor, được dùng để "nhúng" vào các View lớn hơn. Điểm khác biệt so với *View Component* là Partial View không có class C# xử lý logic riêng đi kèm.

2. Cách tạo và sử dụng Partial View:

Bước 1: Tạo file Partial View

- Vẫn theo quy ước chung, thường Partial View được đặt tên bắt đầu bằng dấu gạch dưới _.
- Tạo file Views/Shared/_ProductCard.cshtml với nội dung để render 1 sản phẩm:

```
@model Product

<div class="card">
    <div class="card-body">
        <h5>@Model.Name</h5>
        <p>Giá: @Model.Price $</p>
        <button class="btn btn-primary">Mua ngay</button>
    </div>
</div>
```

Bước 2: Nhúng vào View chính (Index.cshtml)

Dùng `@Html.Partial("_MyPartialView")` hoặc tag helper `<partial name="_MyPartialView" />` để nhúng nó vào một View khác.

Sử dụng Tag Helper `<partial>` để gọi file giao diện này nhiều lần trong vòng lặp:

```
@model List<Product>

<h2>Danh sách sản phẩm</h2>
<div class="row">
    @foreach (var p in Model)
    {
        <div class="col-md-4">
            @* Gọi Partial View và truyền dữ liệu của 1 sản phẩm vào cho nó *@
            <partial name="_ProductCard" model="p" />
        </div>
    }
</div>
```

Sự kỳ diệu: Giao diện của bạn giờ đây cực kỳ sạch sẽ. Nếu sau này có yêu cầu đổi màu nút "Mua ngay", bạn chỉ cần mở đúng file nhỏ `_ProductCard.cshtml` ra sửa 1 lần, lập tức mọi trang web có dùng thẻ sản phẩm này đều được cập nhật đồng loạt!

3.2.3. Tag Helpers - HTML với siêu năng lực

Đây là một trong những tính năng tuyệt vời nhất của ASP.NET Core Razor. Nó cho phép bạn viết code trông rất giống HTML nhưng lại được server xử lý để sinh ra HTML phức tạp hơn.

- **Mục tiêu:** Giúp code trong View sạch sẽ, dễ đọc và an toàn hơn.
- **Ví dụ:**
 - Cách viết truyền thống (dễ sai):

```
<a href="/Home/Index?page=2&category=Soccer">Trang 2</a>
```

Vấn đề: Nếu bạn đổi tên Controller Home thành ProductHome, link này sẽ hỏng.

- Cách viết với Tag Helper (an toàn, chuyên nghiệp):

```
<a asp-controller="Home"
    asp-action="Index"
    asp-route-page="2"
    asp-route-category="Soccer">Trang 2</a>
```

Lợi ích: Code này sẽ được server phân tích. Nếu Controller Home không tồn tại, bạn có thể nhận được cảnh báo. Nó cũng tự động xử lý việc tạo URL một cách chính xác.

- **Một số Tag Helper phổ biến:** asp-controller, asp-action, asp-route-*, asp-for (cho form), asp-validation-for (cho lỗi validation), environment.

3.3. URL Routing - GPS của ứng dụng Web

3.3.1. Routing là gì?

Routing là **hệ thống định vị GPS** của ASP.NET Core. Khi một request với một URL cụ thể được gửi đến, Routing sẽ phân tích URL đó và quyết định chính xác **Controller** và **Action** nào sẽ xử lý nó.

3.3.2. Convention-based Routing (Routing theo quy ước)

Đây là cách cấu hình routing mặc định và phổ biến nhất trong các ứng dụng MVC. Bạn định nghĩa một hoặc nhiều "khuôn mẫu" (template) trong Program.cs, và ASP.NET Core sẽ cố gắng khớp URL của request với các khuôn mẫu đó.

- Cấu hình trong Program.cs:

```
app.MapControllerRoute(
    name: "default",
```

```
pattern: "{controller=Home}/{action=Index}/{id?}";
```

- **Phân tích một "khuôn mẫu":** pattern: "{controller=Home}/{action=Index}/{id?}"
 - {controller=Home}: Đây là một **phân đoạn (segment)** có tên là controller. Dấu = cho biết giá trị **mặc định** là Home nếu URL không cung cấp phần này.
 - {action=Index}: Tương tự, đây là phân đoạn action với giá trị mặc định là Index.
 - {id?}: Đây là phân đoạn id. Dấu ? ở cuối cho biết phân đoạn này là **tùy chọn (optional)**.
- **Ví dụ hoạt động:**

URL người dùng gõ	Controller được khớp	Action được khớp	Id được khớp
/	Home (mặc định)	Index (mặc định)	(không có)
/Products	Products	Index (mặc định)	(không có)
/Products/List	Products	List	(không có)
/Products/Details/5	Products	Details	5

3.3.3. Attribute-based Routing (Routing theo thuộc tính)

Thay vì định nghĩa tất cả các khuôn mẫu ở một nơi, bạn có thể đặt "địa chỉ" trực tiếp trên từng Action.

```
public class ProductsController : Controller
{
    [HttpGet("all-products")] // URL sẽ là /all-products
    public IActionResult List() { ... }
}
```

Cách này rất phổ biến khi xây dựng Web API, chúng ta sẽ tìm hiểu sâu hơn ở Chương 5.

3.3.4. Custom Route (Định tuyến tùy chỉnh)

Custom Route (Định tuyến tùy chỉnh) để làm cho các đường dẫn URL trở nên đẹp mắt và chuẩn SEO hơn (ví dụ: yoursite.com/danh-muc-san-pham thay vì yoursite.com/Products/List).

Trong ASP.NET Core, có **2 cách phổ biến** để tạo Custom Route:

Cách 1: Conventional Routing (Cấu hình tập trung tại Program.cs)

Đây là cách bạn thêm một "bản đồ" mới vào hệ thống định tuyến, báo cho ứng dụng biết rằng khi gặp một chuỗi URL có hình thù cụ thể, hãy bẻ lái nó về một Controller mong muốn.

- **Thao tác:** Mở file Program.cs và thêm đoạn mã app.MapControllerRoute mới **NGAY PHÍA TRÊN** cái route mặc định (default).

```
// 1. CUSTOM ROUTE: Dành riêng cho trang danh mục sản phẩm (SEO URL)
app.MapControllerRoute(
    name: "seo_category",
    pattern: "danh-muc-san-pham",
    defaults: new { controller = "Products", action = "List" });

// 2. DEFAULT ROUTE: (Luôn phải nằm ở dưới cùng)
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

NGUYÊN TẮC: Hệ thống Routing đọc từ trên xuống dưới. Nó giống như rây bột vậy, URL nào lọt qua cái rây đầu tiên (khớp pattern) sẽ được xử lý ngay lập tức. Do đó, **những Route cụ thể (Custom) bắt buộc phải nằm trên Route chung chung (Default)**. Nếu bạn đảo ngược vị trí, Custom Route của bạn sẽ không bao giờ được gọi.

Cách 2: Attribute Routing (Định tuyến trực tiếp bằng Attribute)

Nếu bạn có quá nhiều URL cần tối ưu SEO, file Program.cs của bạn sẽ phình to và rất khó quản lý. ASP.NET Core cung cấp một giải pháp hiện đại hơn: **Attribute Routing**. Cách này cho phép bạn "gắn thẻ" đường dẫn trực tiếp ngay trên đầu mỗi phương thức trong Controller.

- **Thao tác:** Bạn không cần đụng đến Program.cs. Hãy mở trực tiếp file Controller (ví dụ: ProductsController.cs) và thêm [Route("...")].

```
using Microsoft.AspNetCore.Mvc;

namespace YourProjectName.Controllers
{
    public class ProductsController : Controller
```



```
{
    // Gắn cứng URL mong muốn ngay trên đầu Action
    [Route("danh-muc-san-pham")]
    public IActionResult List()
    {
        // Logic lấy danh sách sản phẩm...
        return View();
    }

    // Truyền cả tham số động vào URL (ví dụ: /san-pham/dien-thoai-iphone)
    [Route("san-pham/{slug}")]
    public IActionResult Detail(string slug)
    {
        // Logic tìm sản phẩm theo chuỗi 'slug' (ví dụ: 'dien-thoai-iphone')
        return View();
    }
}
```

Khi nào nên dùng cách nào?

- **Dùng Cách 1 (Program.cs):** Khi bạn muốn định nghĩa một quy tắc chung áp dụng cho hàng loạt trang có cấu trúc giống nhau (ví dụ: trang quản trị Admin `yoursite.com/admin/...`).
- **Dùng Cách 2 (Attribute Routing):** Khi bạn muốn tinh chỉnh chính xác từng URL lẻ tẻ cho mục đích SEO, hoặc khi bạn xây dựng RESTful API. (Đây là cách được các lập trình viên **khuyến dùng nhiều nhất** cho các trường hợp Custom Route).

HƯỚNG DẪN THỰC HÀNH

Các bước:

- Tạo View:
 - Trong HomeController, chuột phải vào tên Action Index -> **Add View...** -> Chọn **Razor View - Empty**. Đặt tên là Index.cshtml.
- Tạo Layout:
 - Trong Solution Explorer, chuột phải vào thư mục Views/Shared -> **Add** ->

New Item... -> Tìm **Razor Layout**. Đặt tên là `_Layout.cshtml`.

– Cấu hình Route:

- Tập trung vào file `Program.cs`, tìm đến dòng `app.MapControllerRoute(...)` để xem và hiểu cấu hình mặc định.

– Code Giao diện:

- Mở các file `.cshtml` và bắt đầu áp dụng các cú pháp Razor, Tag Helper đã học để xây dựng giao diện.

HƯỚNG DẪN DEMO

Bước 1: Tạo view

Khi bạn làm theo hướng dẫn: *Chuột phải vào chữ Index trong HomeController -> Add View -> Đặt tên Index.cshtml*, Visual Studio sẽ không chỉ tạo ra một file bừa bãi. Nó thực hiện một loạt các hành động tự động dựa trên quy ước ngầm định.

1. Quy ước thư mục (Folder Convention)

ASP.NET Core có một quy tắc tìm kiếm file giao diện (View) cực kỳ nghiêm ngặt. Khi Controller có tên là `HomeController` muốn trả về một giao diện, hệ thống sẽ tự động đi tìm theo đường dẫn sau:

- Đầu tiên, nó tìm thư mục gốc mang tên **Views**.
- Tiếp theo, nó tìm một thư mục con mang tên của Controller (bỏ đi chữ "Controller"). Trong trường hợp này là thư mục **Home**.
- Cuối cùng, nó tìm một file `.cshtml` mang tên của Action (phương thức). Ở đây là **Index.cshtml**.

⇒ **Kết quả:** Thao tác "Add View" của bạn sẽ tự động tạo ra cấu trúc thư mục và file chuẩn xác: `Views/Home/Index.cshtml`. Nếu bạn tự tạo bằng tay mà đặt sai tên thư mục (ví dụ ghi nhầm thành `Views/home/index.cshtml`), ứng dụng sẽ báo lỗi không tìm thấy trang ngay lập tức.

2. Mối liên kết giữa Controller và View

Hãy xem lại đoạn code trong `HomeController.cs`:

```
public class HomeController : Controller
{
```

```
public IActionResult Index()
{
    // Khi gọi phương thức View() mà KHÔNG truyền tham số tên,
    // ASP.NET Core tự động hiểu là nó cần tìm file View có
    // cùng tên với Action hiện tại (tức là file "Index.cshtml").
    return View();
}
```

3. Cấu trúc cơ bản của file Index.cshtml vừa tạo

Khi bạn chọn **Razor View - Empty**, Visual Studio sẽ tạo ra một file gần như trống, chỉ chứa một chút thông tin cấu hình ban đầu để bạn bắt đầu viết code HTML và C#.

```
@* Khối lệnh Razor bắt đầu bằng dấu @{ ... } *@
@{
    // Dòng này đặt tiêu đề cho trang, sau này thẻ <title> trong
    _Layout.cshtml sẽ đọc biến này
    ViewData["Title"] = "Index";
}

<h1>Trang chủ của tôi</h1>
<p>Nội dung này được tạo ra từ file Views/Home/Index.cshtml</p>
```

Tóm lại: Thao tác nhỏ này giúp bạn tiết kiệm thời gian tạo cây thư mục thủ công và đảm bảo project của bạn luôn tuân thủ đúng chuẩn kiến trúc MVC của Microsoft. Bạn chỉ việc tập trung vào việc viết code giao diện.

Bước 3: Tạo layout

Khi bạn làm theo hướng dẫn: Chuột phải vào thư mục Views/Shared -> Add -> New Item... -> Chọn Razor Layout -> Đặt tên _Layout.cshtml, bạn không chỉ đang tạo ra một file giao diện bình thường. Bạn đang xây dựng "**bộ khung xương**" (Master Page) cho toàn bộ website của mình. Dưới đây là những nguyên tắc ngầm định cực kỳ thông minh mà ASP.NET Core đang áp dụng.

1. Quy ước thư mục dùng chung (Views/Shared)

Trong kiến trúc MVC, không phải giao diện nào cũng thuộc về riêng một Controller. Những thành phần xuất hiện ở mọi trang như: Thanh điều hướng (Header), Chân trang (Footer), hay menu bên trái (Sidebar) cần được chia sẻ cho toàn bộ ứng dụng.

Hệ thống định tuyến của ASP.NET Core có một cơ chế "tìm kiếm dự phòng" rất

thông minh:

- Khi nó cần tìm một View (hoặc Layout), nó sẽ tìm trong thư mục của Controller hiện tại trước (ví dụ: Views/Home).
- Nếu không thấy, nó sẽ **tự động nhảy vào thư mục Views/Shared** để tìm kiếm.
⇒ **Kết quả:** Việc bạn đặt _Layout.cshtml vào thư mục Shared đảm bảo rằng bất kỳ trang web nào (từ trang Chủ, trang Sản phẩm, đến Giới thiệu) cũng đều có thể tái sử dụng lại bộ khung này mà không cần copy-paste lại code.

2. Ý nghĩa của dấu gạch dưới (_) trong _Layout.cshtml

Bạn có thắc mắc tại sao tên file lại bắt đầu bằng dấu gạch dưới không? Đây là một **quy ước đặt tên** trong thế giới Razor View.

Dấu gạch dưới ngầm thông báo với hệ thống và các lập trình viên khác rằng: *"Đây là một file giao diện nền tảng, không phải là một trang độc lập"*. Người dùng sẽ không bao giờ có thể gõ URL lên trình duyệt kiểu như `yoursite.com/Shared/Layout` để truy cập trực tiếp vào file này được. Nó chỉ được dùng để "lắp ráp" với các file khác.

3. @RenderBody()

Hãy xem lại cấu trúc cơ bản của file _Layout.cshtml vừa tạo. Bạn sẽ thấy một dòng lệnh C# cực kỳ quan trọng: `@RenderBody()`. Đây chính là "trái tim" của Layout.

Nó hoạt động như một cái "khuôn rỗng". Khi người dùng truy cập trang Chủ, ASP.NET Core sẽ làm 2 việc:

1. Dựng lên toàn bộ Header, Footer từ file _Layout.cshtml.
2. Lấy toàn bộ nội dung trong file Index.cshtml của bạn, "nhét" chính xác vào vị trí của hàm `@RenderBody()`.

4. Cấu trúc cơ bản của file _Layout.cshtml

Khi bạn chọn tạo một Razor Layout, Visual Studio cung cấp sẵn cho bạn cấu trúc HTML5 chuẩn của một trang web hoàn chỉnh:

```
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width" />
  <title>@ViewBag.Title - Cửa hàng của tôi</title>
</head>
```

```
<body>
  <header>
    <h2>Thanh Điều Hướng Chung</h2>
  </header>

  <div>
    @RenderBody()
  </div>

  <footer>
    <p>&copy; 2026 - Cửa hàng của tôi</p>
  </footer>
</body>
</html>
```

Tóm lại: Thao tác tạo Layout giúp bạn thiết lập nguyên tắc **DRY (Don't Repeat Yourself - Đừng lặp lại chính mình)** trong thiết kế giao diện. Bạn chỉ cần viết Header và Footer đúng một lần ở Layout, và 100 trang con khác trong dự án sẽ tự động thừa hưởng giao diện này.

Bước 3: Cấu hình route (định tuyến)

Khi bạn mở file Program.cs và tìm đến dòng lệnh `app.MapControllerRoute(...)`, bạn đang chạm tay vào "**trạm điều phối giao thông**" của toàn bộ ứng dụng ASP.NET Core MVC. Khái niệm này được gọi là **Routing**.

1. Tại sao chúng ta cần cấu hình Route?

Khi người dùng gõ một đường dẫn (URL) lên trình duyệt như `yoursite.com/Products`, trình duyệt hoàn toàn không biết Controller hay View là cái gì. Nó chỉ đơn thuần gửi đi một chuỗi văn bản.

Lúc này, ứng dụng ASP.NET Core của bạn cần một "bản đồ" để dịch chuỗi URL đó thành:

- Phải gọi vào class (Controller) nào?
- Phải thực thi hàm (Action) nào trong class đó?
- Có dữ liệu đi kèm (ID) nào không?

⇒ Và `MapControllerRoute` chính là cái bản đồ đó.

2. Phân tích cấu trúc Route mặc định

Hãy nhìn kỹ vào đoạn code được tạo sẵn trong Program.cs:

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Trọng tâm nằm ở chuỗi pattern. Nó chia URL thành 3 khúc (segment), ngăn cách nhau bởi dấu gạch chéo /:

- **{controller=Home}**: Khúc đầu tiên đại diện cho tên Controller. Dấu bằng (=Home) chính là **giá trị mặc định**. Nghĩa là nếu người dùng không gõ khúc này, hệ thống sẽ tự động điền chữ "Home" vào và gọi HomeController.
- **{action=Index}**: Khúc thứ hai đại diện cho tên hàm (Action). Tương tự, nếu không gõ gì, hệ thống sẽ tự động tìm hàm Index().
- **{id?}**: Khúc cuối cùng là dữ liệu truyền vào. Dấu chấm hỏi (?) cực kỳ quan trọng, nó có nghĩa là **tùy chọn (optional)**. Người dùng có thể truyền ID hoặc không, ứng dụng vẫn không bị lỗi.

3. Ví dụ thực tế: Cách Route "bắt" URL

Hãy xem cách hệ thống xử lý các URL khác nhau dựa trên quy tắc mặc định này:

Người dùng gõ URL	Hệ thống tự động dịch thành	Lý do
yoursite.com/Products/Detail/5	ProductsController -> Detail(5)	Khớp hoàn hảo cả 3 khúc.
yoursite.com/Products	ProductsController -> Index()	Thiếu Action, dùng mặc định là Index. Thiếu ID, bỏ qua.
yoursite.com (Trang chủ)	HomeController -> Index()	Trống hoàn toàn. Lấy mặc định Home và Index.

Tóm lại: Khối lệnh cấu hình Route này thiết lập một bộ khung định tuyến thống nhất cho toàn bộ dự án. Nhờ đó, bạn cứ việc tạo thêm hàng chục Controller mới (như CartController, AdminController), chúng sẽ tự động được hệ thống nhận diện và phân luồng URL mà bạn

không cần phải quay lại Program.cs cấu hình thêm bất cứ dòng nào nữa.

Bước 4: Giao diện (Razor & Tag helper)

Đến bước này, bạn đã thiết lập xong "khung xương" (Layout) và "đường đi" (Routing). Giờ là lúc chúng ta thực sự tạo giao diện cho trang web. Khi bạn mở các file .cshtml để code giao diện, bạn đang sử dụng một trong những công cụ Render (tạo hình) mạnh mẽ nhất của Microsoft: **Razor Engine**.

Dưới đây là cách mà cú pháp Razor và Tag Helper phối hợp với nhau để biến những dòng code khô khan thành một giao diện động, có khả năng phản hồi dữ liệu.

1. Cú pháp Razor (@) – Cầu nối giữa HTML và C#

Trong các file .html truyền thống, bạn chỉ có thể viết giao diện tĩnh. Nhưng với .cshtml (C# + HTML), ký tự @ giống như một "công tắc phép thuật".

Bất cứ khi nào trình biên dịch nhìn thấy dấu @, nó hiểu rằng: "À, lập trình viên đang muốn chuyển từ việc viết giao diện HTML sang viết logic C#". Bạn có thể dùng nó để in ra một biến, chạy một vòng lặp foreach, hay kiểm tra điều kiện if/else ngay giữa rừng thẻ HTML.

– **Ví dụ in biến:** <p>Xin chào, @User.Name!</p>

– **Ví dụ vòng lặp:**

```
<ul>
    @* Khởi tạo vòng lặp C# ngay trong HTML *@
    @foreach (var item in Model.Products)
    {
        @* Tại mỗi vòng lặp, in ra một thẻ <li> với dữ liệu động *@
        <li>@item.Name - Giá: @item.Price.ToString("C")</li>
    }
</ul>
```

– Phân tích sự kết hợp giữa HTML và C#

Nhìn vào đoạn code trên, bạn có thể thấy sự lồng ghép cực kỳ mượt mà mà Razor Engine mang lại:

- **@foreach (...):** Báo hiệu cho trình biên dịch biết rằng "Đây là một vòng lặp C#, hãy chạy nó".
- **Thẻ :** Trình biên dịch hiểu đây là HTML tĩnh, nó sẽ giữ nguyên thẻ này.

- **@item.Name** và **@item.Price**: Khi gộp lại dấu @ bên trong thẻ , hệ thống sẽ bóc tách giá trị của biến C# và in nó ra dưới dạng văn bản thuần túy.

Kết quả cuối cùng khi Server gửi về trình duyệt của người dùng sẽ trông như thế này (trình duyệt không hề biết vòng lặp foreach từng tồn tại):

```
<ul>
  <li>Laptop - Giá: $1,500.00</li>
  <li>Chuột không dây - Giá: $25.00</li>
  <li>Bàn phím cơ - Giá: $120.00</li>
</ul>
```

2. Tag Helper – Thẻ HTML mang sức mạnh của Server

Trước đây, để tạo một đường link động trỏ đến một trang chi tiết sản phẩm, bạn phải viết những dòng code lộn xộn trộn lẫn C# và chuỗi URL. Tag Helper ra đời để giải quyết việc đó.

Tag Helper trông giống hệt như các thuộc tính HTML thông thường (như class, id), nhưng chúng lại được xử lý trên Server bởi C# trước khi gửi về trình duyệt. Các Tag Helper mặc định của ASP.NET Core thường bắt đầu bằng chữ asp-.

- **Thay vì viết khó nhìn (cách cũ):** Xem chi tiết
- **Bạn viết bằng Tag Helper (cách mới):** <a asp-controller="Products" asp-action="Detail" asp-route-id="@product.Id">Xem chi tiết

***Sự kỳ diệu:** Nhờ Tag Helper, nếu sau này bạn thay đổi quy tắc định tuyến (Routing) trong Program.cs (như đã nói ở phần SEO URL), bạn **không cần** phải đi tìm và sửa lại hàng trăm thẻ <a> trong các file View. Tag Helper sẽ tự động tính toán và sinh ra đường link /san-pham/... mới nhất và chuẩn xác nhất cho bạn.*

3. Ví dụ: Lắp ráp một file .cshtml hoàn chỉnh

Dưới đây là một ví dụ kết hợp cả HTML Bootstrap, Cú pháp Razor, và Tag Helper trong file Index.cshtml (hiển thị danh sách sản phẩm):

```
@* 1. Khai báo kiểu dữ liệu mà View này sẽ nhận từ Controller (Model) *@
@model IEnumerable<Product>

@{
    // 2. Cấu hình tiêu đề cho Layout
```



```
        ViewData["Title"] = "Danh sách sản phẩm";
    }

<h2 class="mb-4">Sản phẩm nổi bật</h2>

@* 3. Kiểm tra điều kiện bằng Razor *@
@if (!Model.Any())
{
    <div class="alert alert-warning">Hiện chưa có sản phẩm nào trong
kho!</div>
}
else
{
    <div class="row">
        @* 4. Vòng lặp duyệt qua dữ liệu *@
        @foreach (var p in Model)
        {
            <div class="col-md-4">
                <div class="card mb-3 shadow-sm">
                    <div class="card-body">
                        <h5 class="card-title">@p.Name</h5>
                        <p class="card-text text-danger">Giá: @p.Price
$</p>

                        @* 5. Sử dụng Tag Helper để tạo nút điều hướng
chuẩn xác *@
                        <a class="btn btn-outline-primary"
asp-controller="Products"
asp-action="Detail"
asp-route-id="@p.Id">
                            Xem chi tiết
                        </a>
                    </div>
                </div>
            </div>
        }
    </div>
}
```

4. Điều gì xảy ra khi trình duyệt nhận được kết quả?

Một điều cực kỳ quan trọng bạn cần nhớ: **Trình duyệt web (Chrome, Edge, Safari) hoàn toàn không biết C# hay Tag Helper là gì**. Tất cả các lệnh @, vòng lặp foreach, hay thẻ asp-action đều được Server (Kestrel/IIS) chạy ngầm, tính toán, và dịch ra thành HTML thuần túy 100%. Trình duyệt của người dùng chỉ nhận được các thẻ <div>, <a>, <p> thông thường. Điều này giúp mã nguồn C# và cấu trúc backend của bạn được bảo mật tuyệt đối, không bao giờ bị lộ ra ngoài giao diện (Frontend).

CÂU HỎI LÝ THUYẾT

1. Tại sao việc sử dụng "Strongly-typed Models" lại được coi là an toàn và chuyên nghiệp hơn ViewBag?
2. @RenderBody() và @RenderSection() trong một file Layout khác nhau như thế nào?
3. Hãy nêu ra 2 lợi ích chính của việc sử dụng Tag Helper (ví dụ asp-controller) so với việc viết thẻ <a> với href thủ công.
4. Với cấu hình route mặc định ({controller=Home}/{action=Index}/{id?}), URL /Store sẽ được ánh xạ tới Controller và Action nào?

TỔNG KẾT BÀI HỌC

Tuần này, chúng ta đã khoác lên cho ứng dụng một "bộ áo" thực sự. Bạn đã học được:

- **Ngôn ngữ Razor** để kết hợp sức mạnh của C# và HTML.
- Cách xây dựng **giao diện nhất quán** và **tái sử dụng** bằng Layout.
- **Tag Helpers** để giữ cho code View luôn sạch sẽ và an toàn.
- **Routing** để định nghĩa cách người dùng di chuyển trong ứng dụng của bạn thông qua các URL.

Đây là những kỹ năng nền tảng và thiết yếu để xây dựng bất kỳ ứng dụng web nào với ASP.NET Core MVC.

BÀI TẬP VỀ NHÀ

1. **Xây dựng Layout:** Áp dụng kiến thức đã học, thiết kế một file _Layout.cshtml hoàn chỉnh cho dự án SportsStore, bao gồm thẻ header (chứa tên cửa hàng), một div cho sidebar bên trái, một main chứa @RenderBody(), và một footer.
2. Tạo trang "Giới thiệu":

- Trong HomeController.cs, tạo một Action mới tên là About() chỉ đơn giản là return View();.
- Tạo một View tương ứng tên là About.cshtml.
- Trong View này, hãy viết một vài dòng giới thiệu về cửa hàng SportsStore.
- Trong _Layout.cshtml, thêm một link ở header sử dụng Tag Helper (<a asp-controller="Home" asp-action="About">Giới thiệu).
- Chạy ứng dụng và kiểm tra xem bạn có thể điều hướng qua lại giữa trang chủ và trang Giới thiệu hay không, và cả hai trang có dùng chung Layout không.

TÀI LIỆU THAM KHẢO

- [1]. Razor syntax reference for ASP.NET Core: Microsoft Docs - Razor Syntax
- [2]. Layout in ASP.NET Core: Microsoft Docs - Layouts
- [3]. Routing to controller actions in ASP.NET Core: Microsoft Docs - Routing